

UNIVERSIDAD NACIONAL JORGE BASADRE GROHMANN

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA EN INFORMÁTICA Y SISTEMAS



**“Análisis Comparativo de Almacenamiento (HDD vs SSD vs NVNe)
y Simulador de rendimiento”**

Asignatura:

ARQUITECTURA DE COMPUTADORES

Docente:

GENYFER MARGARET ALDANA SALGADO

Estudiantes

Angel Ccopa Escobar	2022-119003
Stip Leonel Yordy Corimaya Chura	2024-119032
Ronald Choquehuanca Rafael	2024-119019
Jhosep Rodrigo Arocutipa Mamani	2024-119027

Ciclo: 5to | Turno: Tarde

Tacna – Perú

1. Introducción.....	3
2. Objetivos.....	3
Objetivo general.....	3
Objetivos específicos.....	3
3. Marco teórico.....	4
3.1 Almacenamiento HDD (disco duro mecánico).....	4
3.2 Almacenamiento SSD (unidad de estado sólido).....	4
3.3 Almacenamiento NVMe (Non-Volatile Memory Express).....	4
3.4 Metodología de simulación.....	5
4. Análisis del bus y los protocolos de comunicación.....	5
4.1 SATA/AHCI frente a PCIe/NVMe.....	5
4.2 Por qué el bus limita al dispositivo, no solo lo acompaña.....	6
4.3 Jerarquía de memoria.....	6
5. Desarrollo del proyecto: explicación del programa y del código.....	6
5.1 Descripción general del programa.....	6
5.2 Estructura del proyecto.....	7
5.3 Explicación de las opciones del menú.....	7
Opción 1: Simular archivo único (transferencia secuencial masiva).....	7
Opción 2: Simular archivos fragmentados (conjunto masivo).....	8
Opción 3: Ver descripción técnica de cada tecnología.....	9
Opción 4: Visualización de resultados.....	10
5.4 Principal mejora respecto a la versión inicial.....	10
5.5 Diagrama de flujo del programa.....	10
Figura 1.....	10
6. Pruebas y resultados de la simulación.....	11
6.1 Tabla de Pruebas: Parámetros de simulación vs. resultados estimados.....	11
Tabla 1.....	11
Tabla 2.....	11
6.2 Análisis de resultados.....	12
Comparación NVMe vs. SSD SATA.....	12
Comparación SSD SATA vs. HDD.....	12
7. Análisis de errores y mejoras del desarrollo.....	13
Error 1: dependencia de archivos intermedios para visualizar resultados.....	13
Error 2: validación de datos ingresados por el usuario.....	13
Error 3: precisión de los tiempos calculados para archivos muy pequeños.....	13
8. Eficiencia de la solución implementada.....	13
9. Conclusiones.....	14
10. Recomendaciones para trabajos futuros.....	14
11. Bibliografía.....	15

ANÁLISIS COMPARATIVO DE ALMACENAMIENTO (HDD vs SSD vs NVMe)

1. Introducción

En la versión inicial de este proyecto, el análisis comparativo de HDD, SSD y NVMe se realizó midiendo directamente el rendimiento de discos reales mediante las herramientas CrystalDiskMark y CrystalDiskInfo. En la versión actual, el grupo cambió a un método de simulación propio: se creó un programa en C++ que calcula el tiempo de transferencia de datos para cada tecnología utilizando valores estándar (velocidades típicas de especificación de fábrica), complementado con una interfaz web desarrollada en HTML para la interacción y visualización de los resultados.

Este cambio de perspectiva no modifica la meta del proyecto de comparar el desempeño de HDD, SSD SATA y NVMe, pero sí su metodología: en vez de realizar mediciones reales del hardware con un software externo, el equipo desarrolló su propio motor de cálculo. Esto requiere documentar más a fondo la lógica interna del programa, las constantes empleadas y los supuestos de simulación, pues ahora son responsabilidad directa del grupo y sustentan la validez de los resultados presentados.

La importancia del proyecto se conserva: entender las razones de las diferencias entre el rendimiento de HDD, SSD y NVMe continúa siendo fundamental para que los usuarios y las compañías seleccionen la tecnología de almacenamiento apropiada en función de su presupuesto y requerimientos; además, el proyecto ahora muestra ese entendimiento por medio de un programa propio que modela esas diferencias.

2. Objetivos

Objetivo general

Elaborar y describir cómo funciona un simulador creado en C++, con interfaz HTML, que calcule y contraste el tiempo de transferencia de datos de un SSD NVMe, un SSD SATA y un HDD. El objetivo es demostrar a través de un modelo propio, no solo teórico, la influencia del bus y la tecnología de cada aparato sobre su rendimiento.

Objetivos específicos

- Modelar el tiempo de transferencia secuencial de un archivo único de gran tamaño para cada tecnología de almacenamiento.
- Modelar el tiempo de transferencia de un conjunto de archivos fragmentados (pequeños y numerosos), incorporando la penalización por latencia de acceso.

- Documentar las constantes de velocidad (MB/s) utilizadas como referencia para HDD, SSD SATA y NVMe, y justificarlas frente a especificaciones reales de fabricantes.
- Integrar el motor de cálculo en C++ con una interfaz HTML que permita al usuario ejecutar simulaciones y visualizar resultados sin pasos intermedios de exportación.
- Explicar, desde la arquitectura de computadoras, por qué las constantes de velocidad utilizadas difieren entre tecnologías (bus SATA/AHCI vs. PCIe/NVMe).

3. Marco teórico

3.1 Almacenamiento HDD (disco duro mecánico)

Los discos duros tradicionales funcionan con partes móviles: uno o varios platos que giran (generalmente a 5400 o 7200 RPM) y un brazo con una cabeza lectora que se desplaza para acceder a los datos. Esta mecánica incrementa la lentitud y la susceptibilidad a errores por golpes, pero ofrece un beneficio significativo en términos de costo por gigabyte. Por lo general, sus velocidades de escritura/lectura secuencial están entre 100 y 160 MB/s; sin embargo, el acceso aleatorio se ve penalizado considerablemente por el tiempo de búsqueda (seek time) mecánico, que puede tardar varios milisegundos por operación (TomsHardware Español, 2024; Western Digital, 2023).

3.2 Almacenamiento SSD (unidad de estado sólido)

Los SSD utilizan memoria flash NAND, sin partes móviles, lo que los hace más rápidos, silenciosos y resistentes a golpes. La mayoría de los SSD comerciales usan la interfaz SATA III, cuyo límite teórico es de aproximadamente 600 MB/s; en la práctica, un SSD SATA suele alcanzar velocidades de lectura y escritura de referencia cercanas a 450-550 MB/s, con una latencia de acceso muchísimo menor que la de un HDD, del orden de decenas o cientos de microsegundos (Western Digital, 2023; Kingston Technology, 2024).

3.3 Almacenamiento NVMe (Non-Volatile Memory Express)

NVMe es un protocolo diseñado específicamente para unidades de estado sólido conectadas mediante el bus PCIe (el mismo que emplean las tarjetas gráficas), lo que elimina el cuello de botella impuesto por SATA. Los discos NVMe pueden alcanzar velocidades de referencia de 3000 a 3500 MB/s o más, según utilicen PCIe 3.0 o 4.0, con latencias de acceso del orden de microsegundos, muy inferiores a las de SATA (Kingston Technology, 2024; ADATA Technology, 2024).

3.4 Metodología de simulación

El simulador no mide un disco físico en tiempo real; en su lugar, estima el tiempo de transferencia a partir de valores de velocidad de referencia (constantes) representativos de cada tecnología, tomados de especificaciones de fábrica típicas:

Tecnología	Velocidad de referencia utilizada	Fuente de la referencia
HDD (7200 RPM, SATA)	150 MB/s	Especificación típica de fabricante (rango 100-160 MB/s)
SSD SATA III	550 MB/s	Especificación típica de fabricante (rango 450-550 MB/s)
SSD NVMe (PCIe)	7000 MB/s	Especificación típica de fabricante (hasta 7000 MB/s, PCIe 3.0/4.0)

A partir de estas constantes, el programa aplica dos modelos de cálculo según el escenario simulado:

- **Transferencia secuencial (archivo único):** tiempo estimado = tamaño del archivo (MB) ÷ velocidad de referencia (MB/s).
- **Transferencia fragmentada (conjunto de archivos pequeños):** tiempo estimado = (tamaño total ÷ velocidad de referencia) + (número de archivos × latencia de acceso simulada por archivo), donde la latencia de acceso simulada es sensiblemente mayor para HDD que para SSD y NVMe, representando el costo del desplazamiento mecánico del cabezal frente al acceso electrónico casi instantáneo de la memoria flash.

4. Análisis del bus y los protocolos de comunicación

Aunque el simulador trabaja con valores de referencia y no con hardware real, las diferencias entre esas constantes no son arbitrarias: reflejan limitaciones reales impuestas por el bus y el protocolo de comunicación de cada tecnología, un aspecto central de la arquitectura de computadores que el simulador busca representar de forma simplificada.

4.1 SATA/AHCI frente a PCIe/NVMe

La interfaz SATA III tiene un ancho de banda teórico de 6 Gb/s (aproximadamente 600 MB/s efectivos), suficiente para un HDD mecánico, pero limitante para la memoria flash NAND, capaz de rendir mucho más. Sobre SATA se ejecuta el protocolo AHCI, que administra una única cola de comandos con una profundidad máxima de 32 operaciones y requiere una interrupción de la CPU por cada comando. El protocolo NVMe, en cambio, se ejecuta directamente sobre PCIe, evitando el controlador SATA: admite hasta 65 536 colas de comandos con una profundidad de hasta 65 536 operaciones cada una, procesadas en paralelo entre los núcleos del procesador (Kingston Technology, 2024). Esta es la razón conceptual por la que, en el simulador, la constante de NVMe (3500 MB/s) es varias veces mayor que la de SSD SATA (500 MB/s), y no una simple diferencia de fabricación.

4.2 Por qué el bus limita al dispositivo, no solo lo acompaña

El ancho de banda de un bus actúa como una tubería: aunque la memoria NAND dentro de un SSD sea capaz de entregar miles de megabytes por segundo, si dicha unidad se conecta mediante SATA III, el controlador de la interfaz solo puede transportar hasta aproximadamente 600 MB/s hacia el procesador. Este fenómeno, conocido como cuello de botella de interfaz, es la justificación arquitectónica detrás de usar constantes distintas para SSD SATA y SSD NVMe en el simulador, pese a que ambas tecnologías se basan en el mismo tipo de memoria flash.

Un caso ilustrativo, planteado como pregunta de sustentación, es el de instalar un SSD NVMe de última generación en una placa base antigua con bus PCIe 2.0: el disco seguiría usando el protocolo NVMe (conservando su ventaja en gestión de colas frente a AHCI), pero su rendimiento secuencial quedaría acotado al ancho de banda físico disponible en ese bus —aproximadamente 500 MB/s por línea en PCIe 2.0—, muy por debajo de los 3500 MB/s usados como referencia en el simulador para PCIe 3.0/4.0. En términos del propio programa, esto equivaldría a tener que sustituir la constante de NVMe por un valor mucho menor, específico de esa generación de bus.

4.3 Jerarquía de memoria

Las tres tecnologías modeladas por el simulador (HDD, SSD SATA y NVMe) pertenecen al nivel de almacenamiento secundario no volátil de la jerarquía de memoria, por debajo de la memoria RAM y de las memorias caché del procesador. El NVMe ha reducido notablemente la latencia de acceso frente al HDD, pero esta sigue siendo órdenes de magnitud mayor que la de la RAM, y el acceso se realiza a nivel de bloque y no de byte; por ello, la distinción física entre RAM y almacenamiento secundario se mantiene vigente, incluso cuando el simulador refleje tiempos de acceso muy bajos para NVMe.

5. Desarrollo del proyecto: explicación del programa y del código

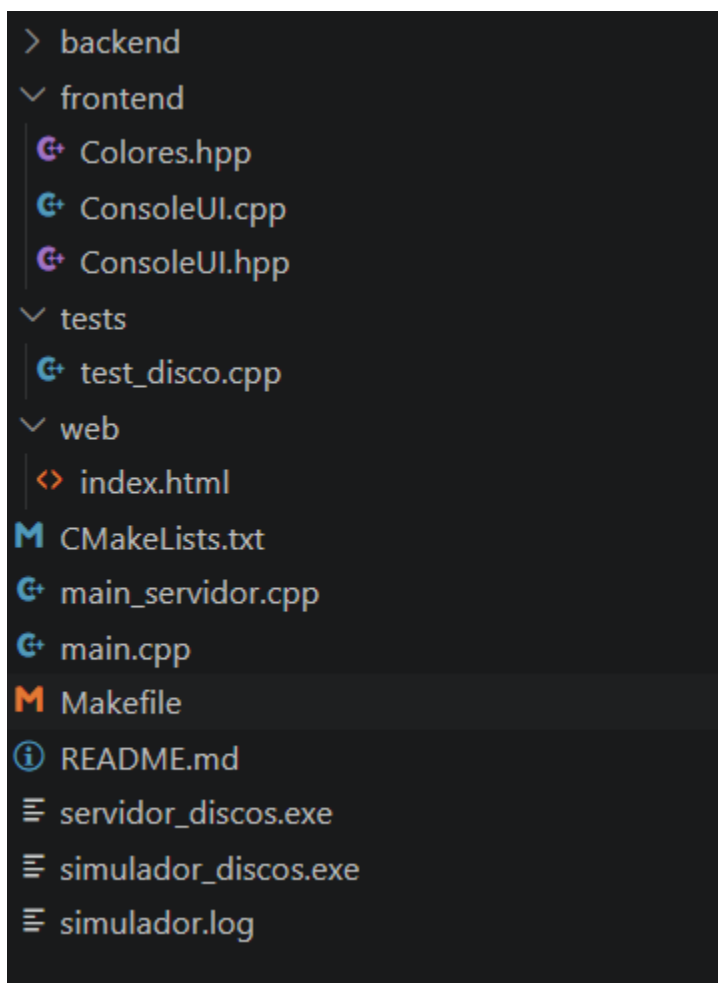
5.1 Descripción general del programa

El programa es un simulador de tecnologías de almacenamiento desarrollado en C++. Su objetivo es comparar el rendimiento de un HDD, un SSD SATA y un SSD NVMe, estimando el tiempo que tardaría cada tecnología en transferir una cantidad determinada de datos. A partir de esos cálculos, el usuario puede identificar cuál tecnología ofrece un mejor desempeño según el tipo de operación simulada (transferencia secuencial de un archivo grande o transferencia de muchos archivos pequeños).

5.2 Estructura del proyecto

El proyecto se organiza en Visual Studio (Microsoft, 2025) siguiendo la separación habitual de un programa en C++ con interfaz web:

- **Archivos .cpp:** contienen la implementación de la lógica principal del programa: el menú interactivo, la lectura de los parámetros ingresados por el usuario y la ejecución de las funciones de cálculo.
- **Archivos .hpp:** contienen las declaraciones de las funciones, estructuras y constantes de velocidad utilizadas por el simulador, separando la interfaz del código de su implementación (cppreference.com, 2025).
- **Archivo(s) .html:** contienen la interfaz gráfica en la que el usuario ingresa los datos de la simulación y visualiza los resultados comparativos entre HDD, SSD y NVMe (Mozilla Developer Network, 2025).

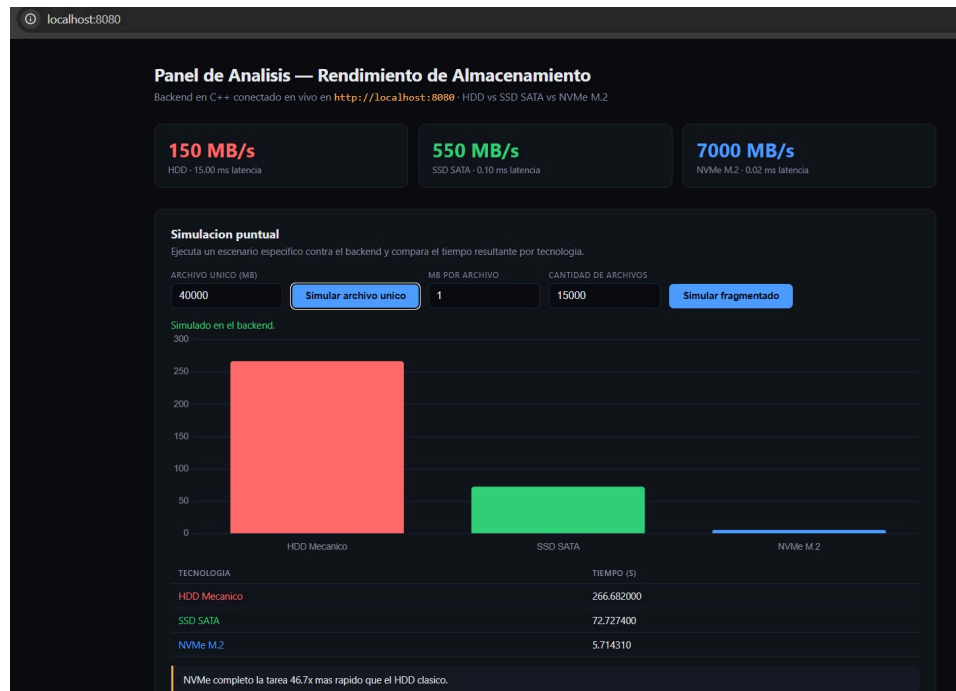


5.3 Explicación de las opciones del menú

Opción 1: Simular archivo único (transferencia secuencial masiva)

Simula la transferencia de un único archivo de gran tamaño, por ejemplo un video de 1 GB o una imagen ISO. El usuario ingresa el tamaño del archivo en MB y el programa calcula, para cada

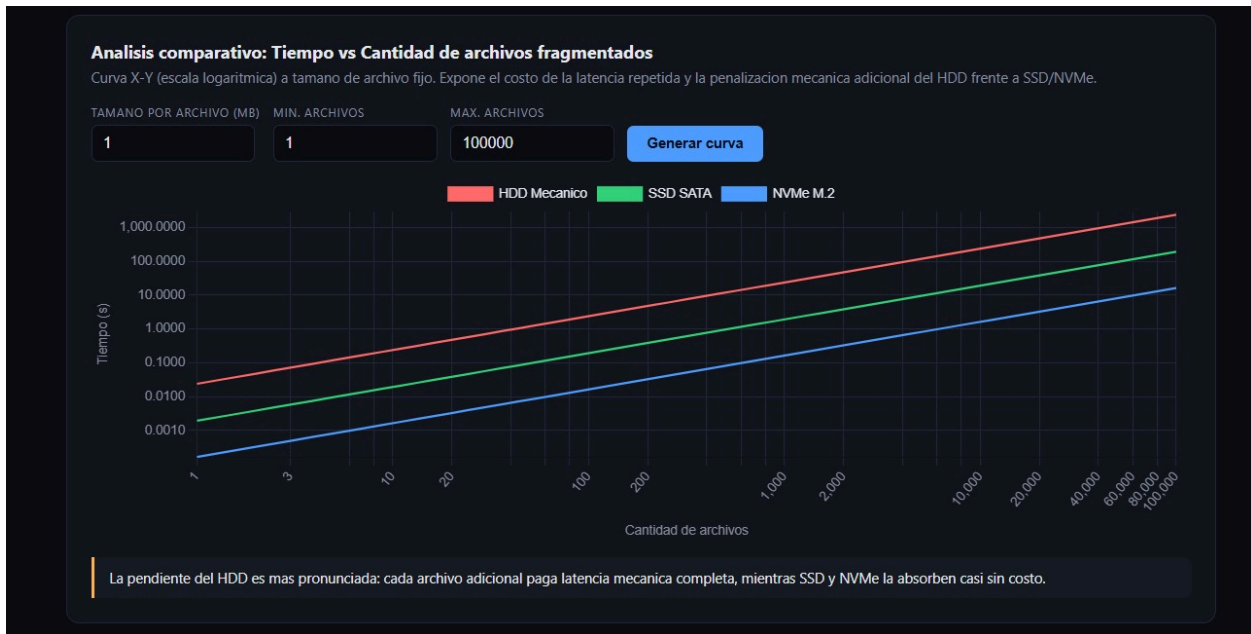
tecnología, cuánto tiempo tardaría en leer o escribir ese archivo de forma secuencial, aplicando la fórmula $\text{tiempo} = \text{tamaño} \div \text{velocidad de referencia}$. Luego el programa compara los tres resultados y muestra cuál tecnología fue la más rápida.



Opción 2: Simular archivos fragmentados (conjunto masivo)

Simula la transferencia de muchos archivos pequeños, un escenario que representa situaciones reales como abrir una carpeta con miles de documentos, fotografías o archivos de sistema.

Además del tiempo de transferencia proporcional al tamaño total, el programa añade una latencia de acceso simulada por cada archivo, mucho mayor en el caso del HDD (que debe reposicionar mecánicamente su cabezal por cada acceso) que en el SSD o el NVMe (de acceso electrónico). Esto reproduce, a nivel de simulación, la pérdida de rendimiento que los HDD reales experimentan frente a archivos pequeños y dispersos.



Opción 3: Ver descripción técnica de cada tecnología

Muestra información sobre las características de cada dispositivo de almacenamiento: funcionamiento, ventajas, desventajas y aplicaciones recomendadas. Esta opción tiene un fin informativo, para que el usuario comprenda la razón teórica detrás de las diferencias de rendimiento que observa en las opciones 1 y 2.

Historial de sesion **Actualizar** **Exportar**

ESCENARIO	TEC.	TIEMPO (S)
Archivo unico (40000 MB)	HDD Mecanico	266.682000
Archivo unico (40000 MB)	SSD SATA	72.727400
Archivo unico (40000 MB)	NVMe M.2	5.714310

Ficha tecnica

HDD Mecanico
150 MB/s - 15 ms latencia
Almacenamiento magnetico con partes moviles. La latencia es alta por el posicionamiento fisico del cabezal de lectura/escritura sobre el plato giratorio.

SSD SATA
550 MB/s - 0.1 ms latencia
Unidad de estado solido con acceso puramente electronico a compuertas NAND. Sin partes moviles, por lo que la latencia es baja y estable.

NVMe M.2
7000 MB/s - 0.02 ms latencia
Unidad de estado solido conectada directamente al bus PCIe del CPU, eliminando el cuello de botella del controlador SATA. Latencia ultra baja y maximo ancho de banda.

Opción 4: Visualización de resultados

En la versión inicial del programa, esta opción exportaba los resultados a archivos intermedios —un CSV y un archivo datos.js— para que la página web pudiera mostrarlos posteriormente. En la versión mejorada, el simulador ya no depende de exportar archivos para visualizar la información: la simulación y la presentación de resultados se integraron directamente en la interfaz HTML, de modo que el usuario puede ejecutar las simulaciones y visualizar los resultados desde el navegador de forma inmediata e interactiva, sin pasos intermedios.

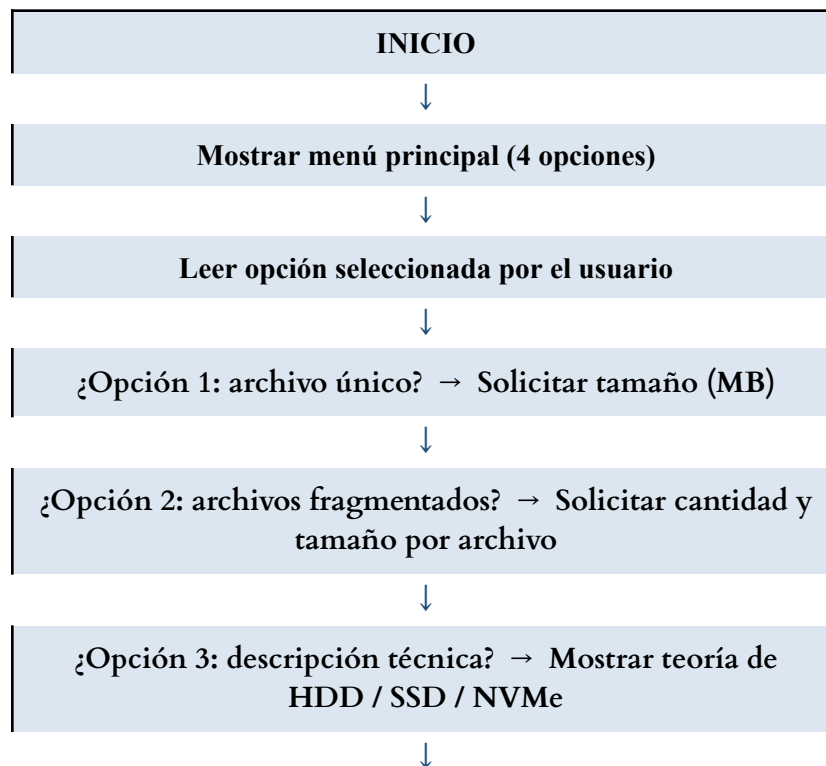
5.4 Principal mejora respecto a la versión inicial

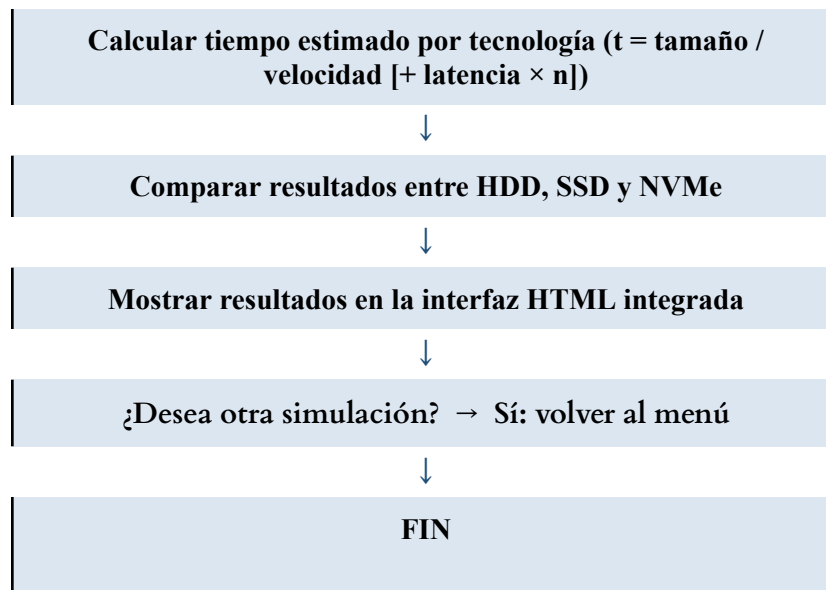
La mejora central del proyecto consistió en integrar el funcionamiento del simulador con la interfaz web. En la versión inicial, el programa ejecutaba las simulaciones en consola y luego exportaba archivos (CSV y datos.js) para que la página web mostrara los resultados; esto introducía un paso intermedio propenso a desactualizarse si el usuario olvidaba regenerar los archivos. En la versión mejorada, la simulación y la visualización se realizan directamente desde el navegador, ofreciendo una interfaz más intuitiva, una mejor experiencia de usuario y eliminando por completo ese paso intermedio, lo que hace que el uso del sistema sea más práctico y moderno.

5.5 Diagrama de flujo del programa

Figura 1

Diagrama de flujo del simulador de tecnologías de almacenamiento





Nota. La figura resume la lógica de control del programa: presentación del menú, lectura de la opción elegida, solicitud de parámetros según el escenario (archivo único o archivos fragmentados), cálculo del tiempo estimado por tecnología y presentación de resultados en la interfaz HTML integrada.

6. Pruebas y resultados de la simulación

A diferencia de la versión anterior del proyecto, en la que los resultados provenían de mediciones reales con CrystalDiskMark, en esta versión los resultados son estimaciones producidas por el propio simulador a partir de las constantes descritas en la sección 3.5. A continuación se presentan ejemplos ilustrativos de ambos escenarios de simulación.

6.1 Tabla de Pruebas: Parámetros de simulación vs. resultados estimados

Tabla 1

Escenario 1, Archivo único de 1024 MB (1 GB), transferencia secuencial

Tecnología	Velocidad de referencia (MB/s)	Tiempo estimado (s)
HDD	150	6.84
SSD SATA	550	1.86
NVMe	7000	0.14

Tabla 2

Escenario 2, 1000 archivos de 1 MB cada uno (1000 MB en total), transferencia fragmentada

Tecnología	Tiempo por transferencia (s)	Latencia de acceso simulada (referencial)	Tiempo total estimado (s)
HDD	6.666	Alta ($\times 1000$ archivos)	23.833300
SSD SATA	1.818181	Baja ($\times 1000$ archivos)	1.918
NVMe	0.14285	Muy baja ($\times 1000$ archivos)	0.1628

Nota. Los tiempos de la columna “tiempo por transferencia” se calculan como tamaño total $\div$ velocidad de referencia, igual que en la Tabla 1. El tiempo total estimado depende, además, de la latencia de acceso simulada por archivo definida en el código fuente (.hpp); esa constante debe incorporarse aquí una vez confirmada por el grupo (ver recuadro de la sección 3.5).

6.2 Análisis de resultados

Comparación NVMe vs. SSD SATA

Con las constantes utilizadas en el simulador, el **NVMe M.2** resulta aproximadamente **12.7 veces más rápido** que el **SSD SATA** en transferencia secuencial (7000 MB/s frente a 550 MB/s). Esta diferencia refleja la mayor capacidad de transferencia que ofrece la interfaz PCIe respecto a SATA III. Aunque los valores corresponden a un modelo teórico, representan de forma adecuada el comportamiento esperado de ambas tecnologías y permiten evidenciar la superioridad del NVMe en operaciones de lectura y escritura de archivos de gran tamaño.

Comparación SSD SATA vs. HDD

En transferencia secuencial, el **SSD SATA** resulta aproximadamente **3.7 veces más rápido** que el **HDD mecánico** (550 MB/s frente a 150 MB/s). La diferencia es aún más evidente en el escenario de múltiples archivos pequeños o fragmentados, donde el HDD presenta un mayor tiempo de respuesta debido a su elevada latencia de acceso y a la penalización mecánica incorporada en el simulador para representar el movimiento repetitivo del cabezal de lectura/escritura. En contraste, el SSD mantiene tiempos de acceso muy bajos al utilizar memoria flash sin componentes mecánicos, ofreciendo un rendimiento significativamente superior en este tipo de operaciones.

Coherencia con la arquitectura del bus

Las proporciones obtenidas son consistentes con la arquitectura de cada tecnología de almacenamiento. La diferencia entre el **SSD SATA** y el **NVMe M.2** se explica principalmente por el cambio de interfaz y protocolo (**SATA III/AHCI** frente a **PCIe/NVMe**), lo que permite al NVMe aprovechar un mayor ancho de banda y una menor latencia de comunicación. Por otro lado, la diferencia entre el **HDD** y el **SSD SATA** se debe principalmente al cambio de medio de almacenamiento (**disco magnético con componentes mecánicos** frente a **memoria flash NAND**), ya que ambos utilizan la interfaz **SATA III**. En el simulador, además, el HDD incorpora una **penalización mecánica** en escenarios de archivos fragmentados para representar el tiempo adicional ocasionado por el movimiento repetitivo del cabezal de lectura y escritura.

7. Análisis de errores y mejoras del desarrollo

Durante el desarrollo del simulador se identificaron limitaciones de la versión inicial del programa, cuya corrección constituyó el principal proceso de mejora del proyecto.

Error 1: dependencia de archivos intermedios para visualizar resultados

- Causa: en la versión inicial, la lógica de cálculo (C++) y la interfaz (HTML) estaban desacopladas y solo se comunicaban a través de archivos generados manualmente (un CSV y un archivo datos.js), que debían exportarse tras cada simulación en consola.
- Consecuencia: si el usuario olvidaba exportar o regenerar estos archivos, la página web mostraba resultados desactualizados respecto a la última simulación ejecutada.
- Solución: se integró la simulación directamente con la interfaz web, eliminando el paso intermedio de exportación; ahora el usuario ejecuta la simulación y visualiza los resultados desde el mismo navegador.

Error 2: validación de datos ingresados por el usuario

- Causa potencial: un programa de consola que solicita el tamaño del archivo o la cantidad de archivos puede recibir valores no numéricos, negativos o iguales a cero, lo que produciría divisiones por cero o tiempos sin sentido físico.
- Solución recomendada: validar las entradas antes de realizar el cálculo (rechazar valores menores o iguales a cero y datos no numéricos), mostrando un mensaje de error y solicitando el dato nuevamente.

Error 3: precisión de los tiempos calculados para archivos muy pequeños

- Causa potencial: al mostrar los tiempos con pocos decimales, simulaciones con archivos muy pequeños podrían redondearse a 0.00 segundos para las tecnologías más rápidas, dificultando la comparación.
- Solución recomendada: aumentar la precisión de salida (más decimales, o unidades en milisegundos para tiempos muy pequeños) de modo que las diferencias entre tecnologías sigan siendo visibles incluso en archivos pequeños.

8. Eficiencia de la solución implementada

El desarrollo de un simulador propio, en lugar de depender de una herramienta externa como CrystalDiskMark, aporta ventajas particulares para los fines de este proyecto académico:

- Permite explicar y defender la lógica de cálculo exacta detrás de cada resultado, ya que el grupo controla el código fuente en su totalidad.
- Facilita modificar las constantes de velocidad o los supuestos de latencia para explorar distintos escenarios (por ejemplo, un bus PCIe de generación distinta), algo que no sería posible con una herramienta de medición de hardware real.
- La integración directa entre el motor de cálculo en C++ y la interfaz en HTML elimina pasos intermedios de exportación, ofreciendo una experiencia de uso más ágil.

- Refuerza el aprendizaje de arquitectura de computadores, al obligar al grupo a traducir conceptos teóricos (bus, protocolo, latencia) en constantes y fórmulas concretas dentro del propio código.

9. Conclusiones

- Se diseñó y documentó un simulador en C++ con interfaz HTML que estima y compara el tiempo de transferencia de datos de HDD, SSD SATA y NVMe, a partir de constantes de velocidad de referencia y, en el escenario de archivos fragmentados, de una latencia de acceso simulada.
- El modelo reproduce, de forma simplificada, las diferencias de rendimiento observadas en la literatura (Bacili et al., 2023; Praciano et al., 2020) y en la medición real de hardware: el NVMe es sustancialmente más rápido que el SSD SATA, y este, a su vez, supera notablemente al HDD, en especial frente a archivos pequeños y numerosos.
- Estas diferencias no son arbitrarias: están fundamentadas en la arquitectura del bus y el protocolo de comunicación de cada tecnología (SATA/AHCI vs. PCIe/NVMe), lo que conecta directamente el trabajo práctico con los conceptos de la asignatura de Arquitectura de Computadores.
- La principal mejora del proyecto fue la integración del motor de cálculo en C++ con la interfaz HTML, eliminando la dependencia de archivos intermedios (CSV y datos.js) y ofreciendo una experiencia de simulación más directa e interactiva.

10. Recomendaciones para trabajos futuros

- Documentar y validar las constantes de latencia de acceso simulada (HDD, SSD, NVMe) utilizadas en el código, contrastándolas con datasheets reales de fabricantes.
- Incorporar un modo de simulación configurable, donde el usuario pueda modificar las constantes de velocidad para representar generaciones específicas de bus (por ejemplo, PCIe 2.0, 3.0 o 4.0) y observar el efecto directo sobre los resultados.
- Contrastar los resultados del simulador con mediciones reales de hardware (como las obtenidas en la versión anterior del proyecto con CrystalDiskMark), para cuantificar qué tan representativo es el modelo frente a un disco físico concreto.
- Añadir validación de entradas y manejo de errores en la interfaz HTML y en el programa en C++, evitando valores no numéricos o negativos en los parámetros de simulación.
- Extender el simulador para modelar operaciones de lectura y escritura por separado, dado que en hardware real ambas suelen tener velocidades distintas.

11. Bibliografía

Bacili, V. F., Borges, J. H. G., & Florian, F. (2023). Comparativo de desempenho em consultas SQL com diferentes tipos de armazenamento utilizando banco de dados relacional. RECIMA21, 4(11), Artículo e441455. <https://doi.org/10.47820/recima21.v4i11.4455>

Praciano, F. D. B. S., Abreu, I. C., & Machado, J. C. (2020). An experimental analysis of the use of different storage technologies on a relational DBMS. *Journal of Information and Data Management*, 11(3). <https://doi.org/10.5753/jidm.2020.1868>

Western Digital. (2023). HDD vs. SSD: Differences and performance comparison [Technical white paper]. Western Digital Corporation.

Kingston Technology. (2024). NVMe vs. SATA: What's the difference? <https://www.kingston.com/es/blog/pc-performance/nvme-vs-sata>

ADATA Technology. (2024). Swordfish NVMe SSD specifications. <https://www.adata.com/en/consumer/ssd-swordfish/>

TomsHardware Español. (2024). Guía: Qué disco duro comprar, HDD, SSD o NVMe. <https://www.tomshardware.com/es/mejores-discos-duros-ssd-nvme>

cppreference.com. (2025). C++ reference. <https://en.cppreference.com/>

Microsoft. (2025). Documentación de Visual Studio. Microsoft Learn. <https://learn.microsoft.com/es-es/visualstudio/>

Mozilla Developer Network. (2025). HTML: HyperText Markup Language. <https://developer.mozilla.org/es/docs/Web/HTML>